

Documentación de Arquitectura y Mapeo UI/UX - Sistema de Contratación

Este documento establece la relación técnica entre las interfaces de usuario (Wireframes) y la arquitectura de software basada en el patrón de tres capas (**Controller, Service, Repository**). En esta fase inicial, se define el comportamiento de los controladores como puertas de enlace (Gateways) para el ruteo de peticiones.

1. Definición de la Arquitectura de Capas

El sistema se estructura en capas independientes para garantizar mantenibilidad y escalabilidad:

1.1 Capa de Controladores (Controller)

- **Función:** Gateway / Punto de Entrada.
- **Responsabilidad:** Recibir peticiones HTTP de la UI, validar parámetros básicos (sintaxis), invocar la lógica en la capa de servicios y retornar una respuesta (JSON/HTML) a la interfaz. No contiene lógica de negocio.

1.2 Capa de Servicios (Service) - *Fase Posterior*

- **Función:** Lógica de Negocio.
- **Responsabilidad:** Validaciones complejas, orquestación de procesos y aplicación de reglas de negocio institucionales.

1.3 Capa de Repositorios (Repository) - *Fase Posterior*

- **Función:** Acceso a Datos.
 - **Responsabilidad:** Comunicación directa con la base de datos (PostgreSQL/MySQL) o sistemas de almacenamiento de archivos (S3/Cloud Storage).
-

2. Mapeo de Interfaces vs. Controladores

A continuación se detalla qué controlador responde a cada pantalla del sistema y los requerimientos funcionales (RF) que atiende.

Pantalla / Interfaz	Requerimientos	Controlador Asociado	Métodos Principales
Login Institucional	RF-09	AuthController	login(), logout(), checkSession()
Panel de Carga (Aspirante)	RF-02, RF-08	ApplicantController	uploadDocument() , deleteDocument()
Envío de Postulación	RF-05	ApplicantController	submitApplication()
Dashboard Admin (Métricas)	RF-11, RF-06	AdminController	getStatistics(), getSystemAlerts())
Listado de Candidatos	RF-07	AdminController	listCandidates() , filterCandidates())
Visor de Expedientes	RF-03, RF-04	ReviewController	getDocumentStream(), saveComment()
Dictamen Final / Modal	RF-10, RF-12	ReviewController	dismissCandidate(), submitFinalVerdict()
Gestión de Roles (RBAC)	RF-01	UserController	createUser(), updatePermissions()

3. Especificación de Endpoints (Lógica del Gateway)

3.1 AuthController

Gestiona la seguridad y el acceso basado en la vigencia de la convocatoria.

- **POST /auth/login**: Autentica al usuario. Si la convocatoria cerró, bloquea el acceso para aspirantes.
- **GET /auth/me**: Retorna el rol del usuario para renderizar la UI correspondiente.

3.2 ApplicantController

Maneja el flujo de entrada de datos del ciudadano.

- **POST /applicant/documents**: Recibe el archivo multipart. El controlador verifica que sea PDF y menor a 5MB antes de enviarlo al servicio.
- **POST /applicant/submit**: Valida que el checklist de documentos esté completo antes de cerrar el folio.

3.3 AdminController

Provee la información para la toma de decisiones.

- **GET /admin/dashboard**: Agregado de datos para los KPIs (Pendientes, Rechazados, Contratados).
- **GET /admin/notifications**: Flujo de eventos en tiempo real sobre cambios de estado.

3.4 ReviewController

El núcleo de la validación técnica.

- **GET /review/file/{fileId}**: Retorna el stream del PDF para el visor embebido (sin descarga forzada).
- **POST /review/comment**: Persiste observaciones asociadas a un documento específico.
- **PUT /review/finalize**: Procesa el dictamen manual. Requiere el flag de confirmación manual del modal.

4. Diagrama de Flujo de Datos

graph LR

A[Interfaz UI] -->|Petición HTTP| B(Controller / Gateway)

B -->|Llamada Funcional| C{Service Layer}

C -->|Persistencia| D[Repository / DB]

D -->|Datos| C

C -->|Respuesta Procesada| B

B -->|JSON/View| A

Referencias

- Documento de Requerimientos Funcionales v1.0.
- Estructura de Wireframes generada en Stitch AI.